

JVM 性能诊断报告

 刷新

线程分析

RUNNABLE: 62 WAITING: 78 TIMED_WAITING: 48

 **潜在问题:** RUNNABLE 线程数量偏高, 可能导致 CPU 使用率飙升或 IO 阻塞。

 **建议:**

- 检查系统 CPU 使用率, 确认是否存在 IO 阻塞或计算密集型任务。

GC分析

 **GC算法:** G1 GC

 **最大暂停时间:** 1310 ms

 **资源警告:** $\text{real} > \text{usr} + \text{sys}$, 表明资源竞争导致 GC 延迟。

 **建议:**

- 确保容器 CPU 资源充足, 设置合理的 CPU request/limit。
- 调整 G1 参数以控制暂停时间。
- 验证 APM Agent 是否干扰。
- 优化 GC 日志存储位置。

内存分析

 **问题嫌疑对象:** TaskThread 保留了大量内存。

 **主要内存消耗:**

- java.util.ArrayList
- org.bson.Document

- java.util.LinkedHashMap
- byte[]

💡 建议:

- 搜索代码中无分页全查、ThreadLocal 未清理等模式。
- 强制启用分页。
- 使用 MAT 的 "Path To GC Roots" 功能确认具体引用链。

综合诊断结论

🟢 **系统状态:** 应用处于健康状态, 但存在隐性资源瓶颈。

🔥 **主要问题:** 容器 CPU 限制过严导致 GC 延迟, 线程可能在等待 IO。

✅ 建议行动:

- 提升容器 CPU 配额至 2 核。
- 验证 APM Agent 影响。
- 优化代码以减少内存泄漏。

📄 原始分析报告

生成时间：2026-03-03_08-46-25

输出内容

THREAD ANALYSIS

以下是对线程转储文件 `threaddump-1755153424580.tdump` 的分析结果:

🔍 一、基础信息

- **分析工具:** 结合了 TDA 官方引擎和自定义 Python 解析逻辑。
- **文件路径:** Linux 系统下的 thread dump 文件。

📊 二、核心摘要 (来自 TDA)

- **总线程数:** 197 个
- **采集时间:** 2025-08-14 06:37:04

- **死锁数量**: 0 →  无死锁

 **注意**: 197 是一个较高的线程数，需结合状态分布判断是否合理。

三、高级线程特征（全部为空）

- **虚拟线程 (Project Loom)** : 未使用 → 应用基于传统线程模型。
- **Native 线程**: 无任何线程处于 native 方法 (如 JNI、socket read) → 所有阻塞均为 Java 层。
- **僵尸线程 (SMR 泄漏)** : 无 → JVM 内存管理正常 (JDK 17+ 特性) 。

 **结论**: 线程模型干净，无底层异常。

四、锁竞争分析

- **无任何线程在等待锁** (即没有 BLOCKED 状态线程) 。
- 所有同步操作要么无竞争，要么使用无锁结构 (如 ConcurrentHashMap) 。

 **结论**: 不存在锁瓶颈，这是非常好的信号！

五、线程状态分布 (关键!)

- **RUNNABLE**: 62
- **WAITING**: 78
- **TIMED_WAITING**: 48

总计: $62 + 78 + 48 = 188$ (注意: $197 - 188 = 9$ 个线程可能为 TERMINATED 或解析遗漏, 但通常可忽略)

状态解读:

- **RUNNABLE**: 62 → 正在运行或就绪 (可能在 CPU 上, 也可能在 OS 队列中)  中等
- **WAITING**: 78 → 无限期等待 (如 Object.wait(), LockSupport.park())  正常
- **TIMED_WAITING**: 48 → 限时等待 (如 Thread.sleep(), Object.wait(1000))  正常

关键观察:

- 62 个 RUNNABLE 线程偏高 (占 ~31%) , 需警惕:
 - 是否 CPU 使用率飙升?
 - 是否存在忙等待 (busy-wait) 或计算密集型任务?
- 126 个等待线程 (78+48) 是典型线程池行为 (如 Tomcat、Dubbo、HikariCP 等空闲 worker) 。

 如果应用是 Web 服务 (如 Spring Boot) , 这种分布 完全正常。

六、热点方法分析

- **没有提取到 RUNNABLE 线程的栈顶方法** → 可能：

1. 所有 RUNNABLE 线程处于 native（但前面 native_threads=[], 矛盾）
2. 调用栈未被正确捕获（如 JIT 优化）
3. 实际在执行 native 代码（如 socket read），但 TDA 未标记为 native

💡 建议：手动检查几个 RUNNABLE 线程的完整栈，确认是否卡在 IO（如 sun.nio.ch.EPoll.wait）。

✖ 七、综合诊断结论

✅ 健康方面：

- 无死锁、无锁竞争 → 并发逻辑安全。
- 无线程泄漏迹象（无僵尸线程）。
- 线程状态以 WAITING 为主 → 符合线程池模型。

⚠ 潜在关注点：

- 62 个 RUNNABLE 线程：需确认是否导致高 CPU。
 - 若 CPU 使用率低 → 可能是 IO 阻塞（如等待网络响应），实际不消耗 CPU。
 - 若 CPU 使用率高 → 需查 热点方法（当前 hot_runnable 为空，建议手动验证）。

📌 建议下一步：

1. 检查系统 CPU 使用率（top 或 htop）：

- 若 CPU 80% → 抓取 CPU 热点线程（配合 top -H + jstack）。

2. 查看 RUNNABLE 线程的完整调用栈：

```
grep -A 10 "java.lang.Thread.State: RUNNABLE" threaddump-1755153424580.tdump | head -30
```

- 如果看到 sun.nio.ch.EPoll.wait → 正常 IO 等待。
- 如果看到业务代码循环 → 可能存在性能问题。

✅ 最终判断

该线程转储显示应用处于健康状态，无并发问题。

高 RUNNABLE 数量很可能是 大量线程在等待网络/磁盘 IO（Java 中仍标记为 RUNNABLE），而非 CPU 瓶颈。

如果你的应用此时 响应缓慢，问题更可能出在：

- 数据库慢查询
- 远程服务超时
- GC 停顿（需 jstat 验证）

- 网络延迟



GC ANALYSIS

结合 GCeasy 报告的分析结果与 JVM 启动参数，我们可以对当前 Java 应用的 GC 行为、性能瓶颈和潜在风险进行深入诊断。整体来看：应用运行稳定（无 fatal 错误），吞吐量极高，但存在资源竞争导致的 GC 延迟问题。以下是逐项分析与优化建议。



一、关键观察总结

维度	现状	评估
GC 算法	G1 GC (-XX:+UseG1GC)	✅ 适合大堆、低延迟场景
堆大小	峰值 2.44 GB（由 -XX:MaxRAMPercentage=75.0 决定）	⚠️ 依赖容器内存总量
吞吐量	99.862%	✅ 极高，说明 GC 开销极小
最大暂停	1310 ms	⚠️ 对延迟敏感型服务偏高
资源警告	3 次 GC 的 $real > usr + sys$	❗ 核心问题：CPU 或 I/O 资源不足
对象晋升率	仅 5 KB/s	✅ 年轻代回收高效，老年代压力极小



二、核心问题：“ $real > usr + sys$ ” 警告



含义

在 Unix/Linux 中：

- $real$ = 实际耗时（墙钟时间）
- $usr + sys$ = CPU 实际工作时间

若 $real > usr + sys \rightarrow$ GC 线程在等待资源（CPU 被抢占、I/O 阻塞、调度延迟）



结合你的 JVM 参数分析原因：

1. 容器 CPU 限制过严（最可能原因）

- 你使用了 `-XX:MaxRAMPercentage=75.0`，说明运行在容器环境（如 Kubernetes）。
- 但未设置 CPU request/limit，或设置过低（如 0.5 核）。
- G1 GC 依赖并发线程（Concurrent Marking）来避免长 STW，但若 CPU 被 throttle，并发阶段变慢，被迫延长 STW（如 Remark、Cleanup）。
- ✅ 证据：Max Pause = 1310ms（典型 G1 Remark/Cleanup 阶段），3 次事件匹配“资源等待”模式。

2. APM Agent 干扰（次要可能）

- -javaagent:/app/apmagent.jar 可能：
 - 注入字节码增加 CPU 开销
 - 触发 System.gc()（尽管 G1 会忽略，但某些 APM 会强制 Full GC）
 - 自身占用 CPU，加剧资源竞争
- 🔍 建议：检查 GC 日志中是否有 Full GC (System.gc())。

3. GC 日志写入 I/O 瓶颈

- -Xlog:gc*...file=/dumps/...：若 /dumps 是网络存储（NFS）或慢速磁盘，日志写入可能阻塞 GC 线程。
- 尤其在高频 GC 时（虽然你吞吐高，但总分配量达 292.2GB，日志量大）。

三、其他指标健康度分析

指标	分析
Throughput 99.862%	说明应用大部分时间在做业务，GC 干扰极小 → G1 配置基本合理
Avg Pause 127.6ms	对批处理/后台任务可接受；对 Web/API 服务偏高
Promotion Rate 5KB/s	几乎无对象存活到老年代 → 年轻代大小可能偏大，可适当调小以降低单次 Young GC 时间
Heap Peak 2.44GB	若容器内存为 3.25GB (2.44 / 0.75)，则容器内存偏小，建议 ≥4GB

四、针对性优化建议

1. 确保容器 CPU 资源充足（最高优先级）

- 在 Kubernetes Pod 中显式设置 CPU：

```
resources:
  requests:
    cpu: "2"
    memory: "4Gi" # 堆约 3GB (75% of 4Gi)，留足 Native/Metaspace 空间
  limits:
    cpu: "2"
    memory: "4Gi"
```

- 💡 为什么？G1 的并发标记需要持续 CPU 时间。2 核可支持 ConcGCThreads=1~2，避免被调度器饿死。

2. 调整 G1 参数以控制暂停时间

```
-XX:MaxGCPauseMillis=300 # 显式声明目标（默认 200ms，但实际常超）
-XX:G1HeapRegionSize=4m # 若常分配 >2MB 对象，避免 Humongous 分配开销
```

-XX:+G1UseAdaptiveIHOP

默认开启，保持即可

- ⚠️ 不要盲目设 MaxGCPauseMillis=100，会导致频繁 GC，反而降低吞吐。

3. 验证 APM Agent 是否干扰

- 临时移除 -javaagent:/app/apmagent.jar，观察：
 - GC 暂停是否下降？
 - real > usr+sys 事件是否消失？
- 若确认干扰，联系 APM 厂商：
 - 关闭自动 System.gc()
 - 降低采样率
 - 使用 -XX:+DisableExplicitGC（谨慎）

4. 优化 GC 日志存储位置

- 将 GC 日志写入本地 SSD，而非共享存储：

```
-Xlog:gc*:file=/tmp/XXXXXXXXXXgc.log:...
```

- 或减少日志级别（生产环境通常不需要 gc+age=trace）：

```
-Xlog:gc*:file=...:time,tags # 移除 trace
```

5. 监控容器资源使用

- 执行以下命令确认是否 CPU throttle：

```
# 查看 CPU throttle
kubectl describe pod <your-pod> | grep -A5 -B5 throttled

# 或进入容器
cat /sys/fs/cgroup/cpu/cpu.stat

# 若 throttled_time > 0，则 CPU 被限制
```

🔥 五、最终建议配置 (JVM + K8s)

JVM 参数（新增/调整）：

```
-XX:+UseG1GC
-XX:MaxRAMPercentage=75.0
-XX:MaxGCPauseMillis=300
```

```
-XX:G1HeapRegionSize=4m
-Dfile.encoding=UTF-8
-XX:MaxDirectMemorySize=512m
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/dumps/[REDACTED]-heapdump.hprof
-Xlog:gc*:file=/dumps/[REDACTED]gc.log:time, tags:filecount=5, filesize=10M
```

Kubernetes 资源:

```
resources:
  requests:
    cpu: "2"
    memory: "4Gi"
  limits:
    cpu: "2"
    memory: "4Gi"
```



结论

当前系统稳定但存在隐性资源瓶颈。1310ms 暂停和 “real > usr+sys” 警告几乎可以肯定是容器 CPU 限制过严所致。优先提升 CPU 配额至 2 核，并验证 APM Agent 影响，即可显著改善 GC 延迟。



MEMORY ANALYSIS



一、明确各视图在 MAT 中的作用

视图 作用 您提供数据中的体现

- Problem Suspect Report: 自动识别最大内存占用嫌疑对象及其引用链
 - "description" + "suspect_objects_by_class" + "all_objects_retained_by_suspects"
- Top Consumers: 列出占用 retained heap 最多的单个对象或类
 - "all_objects_retained_by_suspects" 实际就是 Top Consumers 的展开（按 retained heap 排序）
- Class Histogram: 统计每个类的实例数、shallow/retained heap 总和
 - "all_objects_retained_by_suspects" 中每行的 Objects、Shallow Heap 字段即来自 Histogram



结论：您提供的 JSON 实际上已经融合了 Top Consumers 和 Class Histogram 的核心信息，尤其是 all_objects_retained_by_suspects 列表：

- 按 Retained Heap 降序排列 → 这是 Top Consumers 的特征；
- 包含 实例数量 (Objects) 和 Shallow Heap 总和 → 这是 Class Histogram 的特征。

因此，我们完全可以基于此进行三位一体的交叉分析。

🔍 二、三位一体交叉分析（结合三大视图）

1. Problem Suspect 定位“根因持有者”

- 92 个 TaskThread 保留了 2.84 GB (93.85%) 堆内存。
- 这些线程本身 shallow heap 仅 11KB，说明问题不在线程本身，而在其引用的对象图。

2. Top Consumers 揭示“被保留的最大对象类型”

从 all_objects_retained_by_suspects（即 Top Consumers）看，被这些线程间接保留的最大对象是：

- java.util.ArrayList: ≥2.17 GB
- org.bson.Document: ≥1.79 GB
- java.util.LinkedHashMap: ≥1.78 GB
- byte[]: ≥1.33 GB

💡 Top Consumers 明确指向：大量集合（List/Map）和 MongoDB 文档是内存主体。

3. Class Histogram 验证“异常实例数量”

通过实例数（Objects）判断是否异常：

- CertificationEntity: 393,936 实例 ⚠️ 极高
- org.bson.Document: 1,122,579 实例 ⚠️ 极高
- String: 24,395,800 实例 ⚠️ 极高
- byte[]: 22,054,543 实例 ⚠️ 极高

📊 Class Histogram 证实：业务对象和底层数据结构的数量级远超正常 Web 请求范围，符合“全表扫描”或“无分页批量加载”特征。

🔗 三、构建完整的泄漏路径（Leak Path）

结合三大视图，可推断典型泄漏链如下：

TaskThread (92个)

- └ ThreadLocalMap
- └ Entry[]
- └ Value: RequestContextHolder / UserContext / CustomContext
- └ Holds: List (393k instances)
- └ Each entity holds: TestConfiguration, DescriptAndInstruction
- └ Underlying data from: org.bson.Document (1.1M instances)
- └ Internal: LinkedHashMap + byte[] for strings

或者（若未使用 ThreadLocal）：

TaskThread

- └ Runnable / Callable (e.g., async task or request handler)
- └ Local variable / field: List or List
- └ Not cleared after use → retained until thread dies

⚠ Tomcat 的 TaskThread 是线程池复用线程，生命周期很长。只要它持有一个强引用到大对象，该对象就永远不会被 GC。

✅ 四、修正后的综合结论（真正融合三大视图）

1. Problem Suspect 准确定位到 92 个 Tomcat 工作线程为内存占用根节点；
2. Top Consumers 揭示这些线程保留了巨量的 ArrayList、Document、LinkedHashMap 和 byte[]，总和达 2.84GB；
3. Class Histogram 证实相关类（如 CertificationEntity, Document, String）的实例数量异常高，排除正常缓存可能；
4. 三者共同指向：应用在处理某些请求（或后台任务）时，一次性加载了数十万条 MongoDB 认证数据到内存，并将结果存储在线程生命周期内可达的位置（如 ThreadLocal、未清理的局部变量、静态引用等），导致 GC 无法回收。

🔧 建议行动（更具针对性）

1. 搜索代码中以下模式：

- ⦿ 危险：无分页全查

```
XXXXXXXXXXXXXXXXXXXXXXXXXXXX.findAll();
```

- ⦿ 危险：ThreadLocal 未清理

```
private static final ThreadLocal cache = new ThreadLocal();  
// ... 使用后未调用 cache.remove()
```

- ⦿ 危险：MongoDB 游标未关闭

```
MongoCursor cursor = collection.find().iterator();  
// ... 只遍历部分就 return，未 close()
```

2. 强制启用分页：

```
Page page = repo.findAll(PageRequest.of(0, 100));
```

3. 使用 MAT 的“Path To GC Roots”功能（若您有原始 hprof 文件）：

- 任选一个  Entity 实例 → Merge Shortest Paths to GC Roots → exclude weak/soft references
- 查看是否经过 TaskThread → 确认具体引用链。